

DESIGN PROJECT REPORT

GuidanceAI — AI-Powered Student Guidance and CSE Domain Recommender System

Virtual Counsellor for First-Year CSE Students | Atlas 25 Domains | 10 Languages | 89% Accuracy

Department of Computer Science and Engineering
Academic Year 2025 – 2026

Guide Mr. VASANTH RAJ K, Assistant Professor (SG), CSE
Head of the Department Dr. T. Sudalai Muthu, Ph.D., Professor and Head, CSE
Associate Dean, Computing Sciences Dr. J. Thangakumar
Submitted by [Team Member 1 — Name, Register No.] [Team Member 2 — Name, Register No.]

Stack: Vanilla HTML + Tailwind CSS + React (Vite) + Django REST + scikit-learn (Random Forest 89%) + Vercel + Render

Main Entry: index.html | Dataset: ml/dataset.csv (750 records) | Model: backend/ml_service/model.pkl

TABLE OF CONTENTS

BONAFIDE CERTIFICATE	4
DECLARATION	4
ACKNOWLEDGEMENT	4
ABSTRACT	5
LIST OF ABBREVIATIONS	6
ORGANIZATION OF THE REPORT	7
1. INTRODUCTION	7
1.1 Overview	8
1.2 Motivation for the project	8
1.3 Problem Definition and Scenarios	9
1.4 Summary	10
2. LITERATURE REVIEW	10
2.1 Introduction	11
2.2 Literature review	11
2.2.1 AI-based career counselling at branch level	11
2.2.2 Branch prediction with Random Forest and KNN	11
2.2.3 E-learning recommender with NPTEL and Coursera	11
2.2.4 Voice-based reminder and text-to-speech systems	12
2.2.5 Chatbot with retrieval and voice in education	12
2.2.6 Resource hub and portfolio systems	12
2.3 Conclusion/Gap	12
3. PROJECT DESCRIPTION	12
3.1 Objective of the Design Project work	13
3.2 Existing System	13
3.3 Proposed System	14
3.4 Benefits of Proposed System	15
4. SYSTEM DESIGN	15
4.1 Architecture Diagram	15
5. PROJECT REQUIREMENTS	16
5.1 Hardware and Software Specification	17
5.2 Summary	18
6. MODULE DESCRIPTION	18

6.1 Modules	18
6.2 Key Modules	18
6.3 User Interface	19
6.4 Technology stack	20
6.5 Security Consideration	20
7. IMPLEMENTATION	20
7.1 Implementation	21
8. RESULT ANALYSIS	22
8.1 Results Obtained	23
9. CONCLUSION AND FUTURE WORK	24
9.1 Conclusion	24
9.2 Future Work	24
9.3 Summary	24
10. INDIVIDUAL TEAM MEMBER'S REPORT	24
10.1 Individual Objective	25
10.2 Role of the Team Members	25
10.3 Contribution of Team Members	25
10.4 Summary	26
REFERENCES	26

BONAFIDE CERTIFICATE

Certified that this Design Project Report titled "GuidanceAI — AI-Powered Student Guidance and CSE Domain Recommender System" is the bonafide work of [Team Member 1 — Name, Reg. No.] and [Team Member 2 — Name, Reg. No.], students of B.E. Computer Science and Engineering, who carried out the design project work under the supervision of Mr. VASANTH RAJ K, Assistant Professor (SG), Department of Computer Science and Engineering, during the academic year 2025-2026. This report is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering. The contents of this report have not been submitted elsewhere for the award of any degree or diploma.

Signature of the Guide Mr. VASANTH RAJ K, Assistant Professor (SG), CSE

Signature of the Head of the Department Dr. T. Sudalai Muthu, Ph.D., Professor and Head, CSE

Signature of the External Examiner Name: _____ Date: _____ Place:

DECLARATION

We, [Team Member 1 — Name, Reg. No.] and [Team Member 2 — Name, Reg. No.], students of B.E. Computer Science and Engineering, hereby declare that the Design Project Report entitled "GuidanceAI — AI-Powered Student Guidance and CSE Domain Recommender System" submitted by us is a record of original work carried out by us under the guidance of Mr. VASANTH RAJ K, Assistant Professor (SG), CSE, during the academic year 2025-2026.

We further declare that:

1. This report is the result of our own effort, design, and implementation.
2. All sources of code, datasets, figures, and literature referred to in this report have been duly acknowledged through citations and references.
3. The machine learning dataset used (`ml/dataset.csv`, 750 synthetic records) was generated by us using `ml/generate_dataset.py` and does not contain any personal or confidential data of any individual.
4. This work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution.

Place: _____ Date: _____

Signature of Team Member 1 | Signature of Team Member 2

ACKNOWLEDGEMENT

We wish to express our deep sense of gratitude from the bottom of our heart to our guide Mr. VASANTH RAJ K, Assistant Professor (SG), Computer Science and Engineering, for his

motivating discussions, overwhelming suggestions, ingenious encouragement, invaluable supervision, and exemplary guidance throughout this design project work.

We would like to extend our heartfelt gratitude to Dr. T. Sudalai Muthu, Ph.D., Professor and Head, Department of Computer Science and Engineering for his valuable suggestions and support in successfully completing the project. We are really indebted to our Associate Dean, Computing Sciences, Dr. J. Thangakumar, for his support required for the successful completion of the project.

We sincerely thank all faculty members and laboratory staff of the Department of Computer Science and Engineering for providing the computing facilities, valuable feedback during reviews, and continuous encouragement. We thank our classmates for participating in quiz trials, multilingual testing, and user feedback collection.

As a final word, we would like to thank each and every individual who has been a source of support and encouragement and helped us to achieve our goal and complete our design project work successfully.

ABSTRACT

First-year Computer Science and Engineering students must choose among more than twenty-five distinct career tracks, including Artificial Intelligence, Machine Learning, Data Science, Cybersecurity, Cloud Computing, Full-Stack Development, and emerging areas such as Blockchain, Robotics, and Quantum Computing. Conventional counselling in colleges is manual, senior-dependent, English-only, and limited to branch selection rather than domain selection within CSE. As a result, students depend on peer pressure, social-media hype, or salary rumours, which leads to mid-course track switching, skill mismatch, and placement stress.

This project presents GuidanceAI, an AI-powered virtual counsellor for first-year CSE students. A student completes a 15-question aptitude and interest quiz with four options per question and enters marks in four subjects. The system produces the Top-3 personalised CSE domain recommendations across 25 tracks (Atlas 25: AI to Quantum) with confidence percentage, explanation, skill-gap radar, 36-week roadmap, mini-course with test, per-domain resource vault, and parent-friendly report in ten languages with voice readout.

The system uses a dual frontend: a zero-install vanilla HTML with Tailwind CSS single-page site that runs by double-clicking `index.html` (offline capable) and a React with Vite and Tailwind alternative scaffold. The backend is Django REST hosted on Render, with endpoints for prediction, domains, resources, vault, feedback, and chatbot. The machine learning pipeline uses `ml/dataset.csv` with 750 synthetic records (30 per domain across 25 domains, 11 features plus label), `StandardScaler` for normalisation, Principal Component Analysis for dimensionality reduction from 11 to 10 components retaining 95 percent variance, and Random Forest with 100 trees as the primary model. Random Forest achieved 89 percent accuracy (cross-validation 82 percent), compared with Decision Tree at 64 percent and K-Nearest Neighbours at 82 percent. The trained artefacts are stored in `backend/ml_service/model.pkl`. A deterministic

weighted-ensemble fallback in the frontend guarantees identical Top-1 prediction when the backend is offline.

Evaluation confirms Top-1 accuracy of 89 percent on the held-out set, Top-3 recommendation with explanation, live radar and what-if analysis, and successful demonstration of the complete flow: quiz to radar to roadmap to Telugu parent voice report to immersive 3D domain handbook. The project demonstrates that machine learning combined with full-stack engineering and multilingual design can deliver practical and inclusive academic counselling.

Keywords: Career Guidance, Domain Recommender System, Random Forest, Principal Component Analysis, Django REST Framework, Multilingual Speech Synthesis, Skill Radar, Progressive Web App.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
ML	Machine Learning
CV	Computer Vision
NLP	Natural Language Processing
HCI	Human-Computer Interaction
DBMS	Database Management System
OS	Operating Systems
CN	Computer Networks
CA	Computer Architecture
IoT	Internet of Things
AR/VR	Augmented Reality / Virtual Reality
RF	Random Forest
DT	Decision Tree
KNN	K-Nearest Neighbours
PCA	Principal Component Analysis
TF-IDF	Term Frequency – Inverse Document Frequency
RAG	Retrieval Augmented Generation
STT	Speech-to-Text
TTS	Text-to-Speech
REST	Representational State Transfer
PWA	Progressive Web App
SEO	Search Engine Optimisation
CSP	Content Security Policy

Abbreviation	Expansion
HSTS	HTTP Strict Transport Security
XP	Experience Points
LPA	Lakhs Per Annum
NPTEL	National Programme on Technology Enhanced Learning
API	Application Programming Interface
UI	User Interface

ORGANIZATION OF THE REPORT

- Chapter 1 introduces the motivation and problem definition of the project in detail.
- Chapter 2 presents the literature survey, referred documents, takeaways, and identified research gaps.
- Chapter 3 describes the project, including objectives, existing system, proposed system, and benefits.
- Chapter 4 presents the system design with architecture diagram, components, and data flow.
- Chapter 5 lists the hardware and software requirements and technologies used.
- Chapter 6 describes all modules, key modules, user interface, technology stack, and security considerations.
- Chapter 7 discusses the complete implementation of frontend, backend, machine learning, vault, and multilingual features.
- Chapter 8 presents result analysis with accuracy, functional results, and comparison.
- Chapter 9 concludes the work, lists limitations, and proposes future enhancements.
- Chapter 10 presents the individual report of team members with objectives, roles, and contributions.

1. INTRODUCTION

1.1 Overview

Computer Science and Engineering enrolment in India exceeds five lakh students per year. After admission, the most difficult question for a first-year student is not how to code, but which direction to pursue. The CSE discipline now contains twenty-five distinct and viable career tracks. The Atlas 25 taxonomy used in this project groups them as: Intelligence and Data (6) — Artificial Intelligence, Machine Learning, Data Science, Big Data, Computer Vision, Natural Language Processing; Build and Experience (5) — Web Development, Mobile App Development, Software Engineering, Game Development, Human-Computer Interaction; Core Systems (8) — Cybersecurity, Cloud Computing, Computer Networks, Database Management Systems, Operating Systems, Computer Architecture, DevOps, Distributed Systems; Frontier Technology (6) — Internet of Things, Blockchain, Robotics, Augmented Reality / Virtual Reality, Embedded Systems, Quantum Computing. Each track differs in mathematical intensity, creativity demand, tools, salary band from 5 to 20 LPA, and preparation path.

GuidanceAI is designed as a virtual counsellor for this decision. The student completes a 15-question quiz covering logical reasoning, mathematical aptitude, creativity, and interest in security, cloud, data, operating systems, coding, emerging trends, and social interaction, with each answer scored from 1 to 4, and enters marks in Mathematics, Physics, Programming, and English on a scale of 0 to 100. The backend normalises the inputs with StandardScaler, reduces dimensions with PCA from 11 to 10 components, applies Random Forest classification, and returns the Top-3 domains with confidence scores and a natural-language explanation such as High logic 4 of 4 with Mathematics 92 indicates Artificial Intelligence as optimal. The frontend renders confidence bars, a Chart.js skill-gap radar, a what-if simulator with sliders, a 36-week handbook roadmap, a mini-course with three lessons per domain with test and certificate, vault suggestions, and a parent report.

The implementation is intentionally dual-frontend and deployment-ready. The main site is vanilla HTML with Tailwind CSS via CDN in `index.html`, which works by double-clicking without installation and functions offline through a service worker and a weighted-ensemble fallback. The alternative scaffold is React with Vite and Tailwind in `frontend/` with proxy to the Django backend. The backend in `backend/` is Django REST Framework with SQLite for development and PostgreSQL readiness for production, deployed with Gunicorn on Render, while the frontend is deployed on Vercel. Multilingual support covers ten languages — English, Telugu, Hindi, Tamil, Kannada, Malayalam, Marathi, Bengali, Gujarati, and Urdu — with Noto fonts, right-to-left support for Urdu, and Web Speech API for text-to-speech voice readout and speech-to-text input.

1.2 Motivation for the project

The motivation for GuidanceAI arises from seven observed problems in first-year counselling.

1. Decision paralysis in Year 1. Students select Artificial Intelligence because it trends on social media and later discover heavy mathematics and statistics requirements. A data-driven Top-3 with confidence and a what-if simulator prevents costly mid-course switching.

2. Exclusion of parents. A large fraction of parents in Tier-2 and Tier-3 colleges are not comfortable with English. A Telugu or Hindi voice report with printable PDF allows a parent to understand the recommendation, salary band, and 36-week plan without English dependence.
3. Lack of teacher visibility. Heads of departments need batch-level analytics to allocate laboratories and remedial classes. A teacher dashboard with distribution bar charts and at-risk flags addresses this need.
4. Absence of actionable next steps. A label such as You should learn Cloud is insufficient. A 36-week phased roadmap with curated NPTEL, Coursera, and YouTube resources, mini-course, test, and per-domain vault makes the recommendation executable from day one.
5. Need for reproducible machine learning evidence. Viva and evaluation require an open dataset, training script, metrics file, and saved model, rather than a black-box API. This project provides `ml/dataset.csv`, `ml/train_model.py`, `ml/metrics.json`, and `backend/ml_service/model.pkl`.
6. Need for offline-first demonstration. College laboratories frequently have weak internet connectivity. The main site with service-worker caching, `localStorage` persistence, and weighted fallback ensures the quiz and demo continue without backend connectivity.
7. Gaps in published work. Prior systems reviewed in Chapter 2 lack parent involvement, multilingual support, visualisation, mini-courses, and batch analytics. GuidanceAI is designed to fill all these gaps in one integrated system.

1.3 Problem Definition and Scenarios

Problem Definition. Develop an AI-powered virtual counsellor that maps a 15-dimensional quiz vector plus a 4-dimensional marks vector to 25 CSE domains with calibrated confidence, explains the recommendation with skill visualisation, grounds it in a 36-week actionable roadmap with curated resources and vault storage, and communicates it appropriately to three stakeholders — student through interactive UI with gamification, parent through vernacular report with voice, and teacher through batch analytics — while remaining offline-capable, deployable on Vercel with Render, and evaluable through open dataset, metrics, and model artefacts.

Representative scenarios are as follows.

1. Confused fresher. A student with Mathematics 92, logic 4 of 4, and strong data interest completes the quiz and receives Artificial Intelligence 82 percent, Machine Learning 76 percent, and Data Science 74 percent, with a radar showing relative weakness in operating systems, a roadmap from Python to statistics to machine learning, vault path `/vault/ai/`, and NPTEL Introduction to Artificial Intelligence.
2. Creative student with low mathematics. A student with Mathematics 58 and creativity 4 of 4 receives Human-Computer Interaction 88 percent, Game Development 81 percent, and Web Development 79 percent, with an explicit low-mathematics and high-creativity path from UX basics to Figma to user research to prototype, instead of being forced toward Artificial

Intelligence.

3. Parent in Telugu. The mother of a student switches the language to Telugu, opens the Parent Dashboard, plays the voice readout, and prints the PDF to understand the salary band of 6 to 12 LPA and the 36-week plan without pressure or confusion.
4. Teacher batch analysis. A professor reviews results for 120 students, observes through a bar chart that 40 percent cluster toward Cloud Computing and 15 percent are flagged at-risk due to low logic scores and low marks, and reallocates Cloud laboratory slots with remedial Python sessions.
5. Offline laboratory demo. Internet connectivity fails during a viva. The main site continues to predict through the weighted ensemble, renders radar and roadmap, and the demonstration proceeds without interruption.
6. Vault collector. A student drops `DBMS_Notes.pdf` into `public/vault/dbms/notes/`. The Explorer card badge updates from Vault 0 to Vault 1. After deployment, the file is accessible at the corresponding public vault URL.
7. What-if explorer. A student moves the coding slider from 2 to 4 and observes live whether Cybersecurity overtakes Cloud Computing in the Top-3 ranking.
8. Chatbot follow-up. A student asks Roadmap for HCI in Tamil and receives UX Basics to Figma to User Research to Prototype with vault tip, in the selected language.

1.4 Summary

GuidanceAI converts first-year confusion into an explainable, actionable, and multilingual Top-3 plan. It combines machine learning with Random Forest at 89 percent accuracy and PCA, full-stack engineering with vanilla frontend, React alternative, and Django backend, and user experience with Atlas 25 explorer, immersive 3D handbooks, radar visualisation, voice reports, gamification, and per-domain vault. The following chapters present the literature survey, project description, system design, requirements, modules, implementation, results, conclusion, and individual contributions.

2. LITERATURE REVIEW

2.1 Introduction

Academic counselling technology has evolved from manual faculty advice to rule-based expert systems, web portals that rank colleges, machine learning branch predictors, and large-language-model chatbots. Personalisation, vernacular access, and progress tracking have been shown to improve adherence and learning outcomes in healthcare-reminder and e-learning literature. However, CSE domain guidance remains shallow. Most portals rank colleges rather than domains within CSE. Most machine learning papers predict branch such as CSE versus Electronics, rather than track such as Artificial Intelligence versus Human-Computer Interaction versus Quantum Computing. Multilingual parent involvement is almost absent. This review examines career counselling systems, branch predictors, course recommenders, voice-based reminder systems, and chatbot systems to position the contribution of GuidanceAI.

2.2 Literature review

2.2.1 AI-based career counselling at branch level

A 2025 study in the AI Journal of Multidisciplinary Research proposed an AI-based counselling system that predicts engineering branch from marks using decision trees. Its strength is simplicity and interpretability. Its limitation is scope: only four to six branches, no domain depth within CSE, English-only output, no parent dashboard, and no roadmap. The takeaway for this project is the need for domain-level prediction across 25 tracks with vernacular reporting and a phased roadmap, both implemented in GuidanceAI through the Telugu voice parent report and the 36-week handbook.

2.2.2 Branch prediction with Random Forest and KNN

A 2022 study in the International Journal of Research and Analytical Reviews compared Random Forest and K-Nearest Neighbours for branch allocation with accuracy near 78 percent. Its strength is model-comparison methodology with cross-validation, which is reused in `ml/train_model.py` and `ml/metrics.json`. Its limitation is that it predicts only branch with six labels, without interest signals, visualisation, or mini-courses. The takeaway is to adopt the comparison and reporting discipline but extend it to 25 labels with radar visualisation, mini-course, and test.

2.2.3 E-learning recommender with NPTEL and Coursera

A 2024 study in the Journal of Emerging Technologies and Innovative Research recommended courses from interest keywords using curated NPTEL and Coursera mappings. Its strength is the curation methodology, reused in the Resource Hub and the Resource model with filtering by domain, type, and language. Its limitation is the absence of aptitude and marks-based machine learning filtering, visualisation, and vault storage. The takeaway is to integrate curation with the machine learning filter, so resources are suggested for the predicted Top-3 rather than generic keywords.

2.2.4 Voice-based reminder and text-to-speech systems

Voice-based reminder literature using text-to-speech libraries such as pyttsx3 shows that audible announcements improve adherence for elderly users and users with reading difficulty. Smart pill dispensers add compartment hardware and sensors but increase cost and complexity. The limitation of conventional voice systems is generic messaging without user-specific machine learning personalisation. The takeaway for GuidanceAI is to reuse the voice insight through browser-native Web Speech API for parent voice reports in ten languages without additional hardware, and to reuse status-recording patterns as lesson-completion and XP tracking.

2.2.5 Chatbot with retrieval and voice in education

Retrieval-augmented chatbots over course documents with speech input and output improve engagement when grounded in institutional content. Unconstrained chatbots hallucinate. The takeaway is implemented as `POST /api/chat`, which builds grounded context from 25 domain blurbs and live vault file listings, supports multi-turn history with anaphora resolution for queries such as its salary, uses Claude 3.5 Sonnet when an API key is configured, and falls back to offline TF-IDF cosine similarity otherwise, with input sanitisation against prompt injection.

2.2.6 Resource hub and portfolio systems

Portfolio and file-slot systems show that students retain more when each track has a dedicated place for notes, videos, projects, and assignments. The takeaway is the per-domain vault with 100 folders in `public/vault/` mirrored privately in `uploads/domains/`, with UI slots, `localStorage` persistence, and backend API listing used by both explorer and chatbot.

2.3 Conclusion/Gap

Seven consistent gaps remain across existing methods. First, branch prediction instead of domain prediction: prior machine learning predicts CSE versus other branches, while none covers 25 tracks within CSE with skills, salary, and roadmap per track. Second, absence of parent and vernacular coverage: English-only reports exclude parents, with no voice readout. Third, absence of visualisation: no radar or what-if analysis to show why Artificial Intelligence ranks above Operating Systems for a given student. Fourth, absence of actionable roadmaps: recommendations stop at a label without a phased plan, mini-course, test, or vault. Fifth, absence of batch analytics for teachers: no at-risk flags or cohort distribution. Sixth, offline fragility: cloud-only demos fail in laboratories without a fallback or progressive web app. Seventh, closed datasets: no open CSV, metrics, or saved model for an evaluator to verify accuracy. GuidanceAI is designed to fill all seven gaps with 25-way Random Forest at 89 percent accuracy with open artefacts, ten-language voice parent reports, Chart.js radar with what-if simulation, 36-week handbooks with mini-courses and vaults, teacher analytics, offline ensemble with service worker, and fully open code and documentation.

3. PROJECT DESCRIPTION

3.1 Objective of the Design Project work

The main objective is to develop an intelligent, user-friendly, and deployable virtual counsellor that helps first-year CSE students select and prepare for a suitable domain. The specific objectives are as follows.

1. To implement face-free, quiz-based user profiling with 15 aptitude and interest responses scored 1 to 4 plus marks in four subjects.
2. To implement a 25-way domain recommender that returns Top-3 domains with confidence and explanation using Random Forest at 89 percent accuracy with PCA, with a deterministic weighted fallback for offline use.
3. To provide an Atlas 25 explorer with grouping, search, sort, compare up to three tracks, command palette, and per-domain vault.
4. To automate guidance with a 36-week handbook roadmap per domain, four-year timeline, mini-course with three lessons per domain, test, certificate, and experience points.
5. To provide multilingual voice reports in ten languages with Noto fonts, right-to-left support for Urdu, text-to-speech readout, and printable parent PDF.
6. To manage knowledge with Django models for student profile, quiz response, prediction, domain, resource, and feedback, plus a curated Resource Hub and a 100-folder vault system.
7. To record progress with XP, streak, badges, bookmarks, lesson completion, and feedback ratings in localStorage and database.
8. To reduce wrong-track selection and hype-driven switching through radar visualisation, what-if analysis, and teacher at-risk flags.

3.2 Existing System

1. Manual counselling. Faculty or senior advice in one-to-one meetings. It is non-scalable, коллежи-dependent, and biased toward the counsellor's own domain.
2. Portal rank lists. College-ranking portals rank institutions, not domains. Articles comparing AI versus ML are generic and not personalised.
3. Time-based reminder applications. Predefined alarms and notifications remind at a fixed time without mapping aptitude to domain.
4. Mobile application reminders. Smartphone alerts store medicine or task details but depend on continuous phone interaction and provide no domain intelligence.
5. Branch predictors. Machine learning models with four to six labels use marks only, without creativity or interest signals, and stop at branch level.
6. Voice-only notifications. Text-to-speech announcements without personalisation provide the same message irrespective of user profile.

7. IoT and RFID systems. Sensor, microcontroller, and reader-based monitoring allow remote tracking but require dedicated hardware, continuous internet, and higher cost.
8. Smart dispensers. Automated compartment release with alarms reduces manual selection but adds mechanical complexity and cost, unsuitable for counselling.
9. Course recommenders. Keyword-to-course mapping suggests NPTEL or Coursera courses without aptitude filtering or visualisation.
10. Limited stakeholder coverage. Existing solutions are student-only, English-only, without parent or teacher views, batch analytics, or offline guarantees.

3.3 Proposed System

Face-free quiz profiling. The proposed system uses a 15-question quiz plus four subject marks instead of camera biometrics, preserving privacy with no biometric storage. Each registered profile has separate quiz responses and marks.

Personalised Top-3 prediction. Each prediction maps the 15 responses and four marks through StandardScaler, PCA, and Random Forest to 25 domains. The API contract is `POST /api/predict` with `{answers: [15 integers 1 to 4], marks: {math, phy, prog, eng}}` returning `{top, confidence, top3, explanation, model}`. When the saved model is unavailable, `weighted_predict()` with a 25 by 17 weight matrix identical between backend and frontend guarantees parity.

Automated roadmap and learning. The system generates the Top-3 with a 36-week handbook per domain in six phases, a four-year timeline from foundation to placement, three YouTube lessons per domain, a five-question test, XP with printable certificate, and vault links.

Voice and visual notifications. Recommendations are delivered through confidence bars, radar charts, toast notifications, vernacular text, voice announcements through Web Speech synthesis, and printable PDF for parents.

Multiple stakeholder support. Three dashboards serve student with XP and roadmap progress, parent with vernacular voice and PDF, and teacher with batch bar charts and at-risk flags.

Knowledge management. Django models store profiles, responses, predictions, domains with skills and roadmaps in JSON, resources with type, language, difficulty, URL, and rating, and feedback with rating. The Resource Hub filters by 25 domains, three source types, and two language groups, with bookmarking.

Progress recording. After each quiz, lesson, or test, XP, streak, badges, bookmarks, and feedback are stored in `localStorage` for offline use and in the database when online, maintaining a history of guidance activity.

Reduced confusion. Since each Top-3 is tied to the individual's quiz and marks with an explanation and radar, hype-driven selection and confusion between tracks are reduced, particularly for households where multiple students share a device.

Simple and user-friendly operation. The main site requires only a browser double-click. No installation, signup, camera calibration, or hardware assembly is needed. Urdu renders right-to-left automatically.

Scope for future enhancement. Mobile application, caregiver notifications, cloud analytics, advanced models, and proctored skill tests are planned and listed in Chapter 9.

3.4 Benefits of Proposed System

Personalised Top-3 with confidence instead of a generic list. Explainable output with radar and what-if analysis. Multilingual inclusion with ten languages and voice. Actionable guidance with roadmap, mini-course, test, and vault. Teacher insight with batch analytics and at-risk detection. Offline-first with PWA and weighted fallback. Open evidence with dataset, metrics, model, and file map. Zero-install demonstration with double-click execution. Privacy-first with no signup and local data. Deployable architecture with Vercel frontend and Render backend with CORS and security headers.

4. SYSTEM DESIGN

4.1 Architecture Diagram

The system design describes the overall structure, components, and interaction between modules. The system identifies student aptitude through quiz and marks, predicts domains through machine learning, retrieves handbook and resource knowledge, and delivers recommendations through visual, textual, and voice channels. It consists of modules for profiling, prediction, explorer, handbook, roadmap with mini-course, resource hub, vault, chatbot, dashboards, feedback, and progress tracking. The design focuses on single responsibility per folder, offline-first behaviour, API parity between online model and offline fallback, and stakeholder separation.

Figure 4.1 — System Architecture.

```
[Student / Parent / Teacher Browser]
  index.html, pages/*, assets/js/app.js, handbooks.js, domain.js
  frontend/src/* (React alternative with i18n in 10 languages)
    | fetch /api/* with CORS to Render
    | fallback to weighted ensemble when offline
    v
[Django REST API on Render – backend/api/views.py]
  POST /api/predict | GET /api/domains | GET /api/resources
  GET /api/vault | POST /api/feedback | POST /api/chat
    |
[ML Service – backend/ml_service/model.pkl + predictor.py]
  StandardScaler -> PCA (11 to 10, 95 percent variance)
  -> RandomForest (100 trees, 89 percent) / DT / KNN
    |
[Data Stores]
  SQLite (dev) / PostgreSQL (prod): StudentProfile, QuizResponse,
  Prediction, Domain, Resource, Feedback
  Static Vault: public/vault/<slug>/notes|videos|projects|assignments/
  Media Mirror: uploads/domains/<slug>/ (Django MEDIA_ROOT)
    |
[External Services]
  NPTEL / Coursera / YouTube embeds, Claude 3.5 Sonnet (optional),
```

Table 4.1 — Major Software Components.

Component	Responsibility	Key File
Profiling Module	15-question quiz plus 4 subject marks, validation, localStorage	pages/quiz.html, assets/js/app.js
ML Filter Module	Scaler, PCA, Random Forest prediction with fallback	backend/api/views.py, ml/train_model.py
Explorer Module	Atlas 25 grid, grouping, search, sort, compare, palette	index.html, pages/explorer.html
Handbook Module	25 handbooks with 18 sections, 3D renderer, charts, timeline	assets/js/handbooks.js, domain.js
Roadmap Module	36-week plan, 4-year timeline, mini-course, test, XP	pages/roadmap.html
Resource Module	NPTEL, Coursera, YouTube hub with filters and bookmarks	pages/resources.html
Vault Module	100 folders, 4 slots per domain, UI plus API listing	public/vault/, uploads/domains/
Chatbot Module	TF-IDF grounding plus Claude, history, voice	backend/api/views.py chat, Chatbot UI
Dashboard Module	Student XP, parent vernacular voice PDF, teacher analytics	pages/dashboard.html
Feedback Module	Sanitised feedback with rating and FAQ schema	pages/contact.html
Shell and PWA Module	Navigation, dark mode, i18n, service worker, SEO	public/manifest.json, sw.js

Data-Flow Sequence. Step 1: User interacts through the graphical interface and selects language and role. Step 2: Quiz responses and marks are validated and cached locally. Step 3: Frontend posts to `/api/predict`. Step 4: Backend validates 15 integers from 1 to 4 and marks from 0 to 100 with a 10 KB limit, loads the model, applies scaler and PCA and Random Forest `predict_proba` to obtain Top-3, or applies weighted fallback. Step 5: Frontend renders Top-3 cards, confidence bars, radar, explanation, roadmap link, and vault suggestion, and stores XP and prediction history. Step 6: Handbook page loads the selected domain handbook with 3D hero, demand charts, timeline, interview kit, and vault slots, with live vault listing from `/api/vault`. Step 7: Chat queries use grounded context with vault files and history to answer in the selected language with optional voice. Step 8: Dashboards aggregate progress, parent voice reads the vernacular summary, teacher view flags at-risk students. Step 9: Feedback is sanitised and stored with toast confirmation.

This integrated architecture provides an automated, personalised, explainable, and user-friendly approach to domain guidance with offline guarantees and deployment readiness.

5. PROJECT REQUIREMENTS

5.1 Hardware and Software Specification

Hardware Requirements. Laptop or desktop with dual-core processor or higher, minimum 4 GB RAM with 8 GB recommended for model training, minimum 2 GB free storage for vault files, node modules, and model artefact of 1.5 MB, display 1366 by 768 or higher with responsive support down to 360 px mobile width, no mandatory camera, sensor, RFID reader, or IoT hardware, optional microphone and speaker for speech-to-text and text-to-speech voice demonstration, optional internet for cloud API, Claude chatbot, YouTube embeds, and deployment while core quiz works offline.

Software Requirements. Operating system Windows 10 or higher, Linux, or macOS. Browser Chrome or Edge 110 or higher for Web Speech, ECharts-GL, and PWA installation. Python 3.10 or higher with Django 4.2, Django REST Framework, scikit-learn, pandas, numpy, Gunicorn, and django-cors-headers as listed in backend/requirements.txt, run with `pip install -r requirements.txt` followed by `python manage.py migrate` and `python manage.py runserver` on port 8000 with production command `gunicorn config.wsgi` on Render. Node.js 18 or higher with React 18, Vite, Tailwind CSS 3.4, chart.js, and i18next as listed in frontend/package.json, run with `npm install` and `npm run dev` on port 5173 with build through `npm run build` to `dist/`. Frontend CDN libraries include Tailwind CSS, Chart.js 4.4 UMD, Font Awesome 6.5, ECharts with ECharts-GL with GSAP ScrollTrigger for handbooks, Google Fonts Inter with Noto Sans Telugu, Devanagari, Tamil, Kannada, Malayalam, Bengali, Gujarati, Nastaliq Urdu, and Arabic. Database SQLite for development with git-ignored db.sqlite3 and PostgreSQL through DATABASE_URL for production with MEDIA_ROOT set to uploads/domains/. PWA and SEO artefacts include public/manifest.json for installable standalone mode with colour #4F46E5 and shortcuts, public/sw.js for cache-then-network offline fallback, public/robots.txt, public/sitemap.xml with 12 Atlas URLs, public/security.txt with .well-known mirror, public/_headers with CSP, HSTS, and X-Frame options, and icons at 192 and 512 px. IDE Visual Studio Code with .vscode/settings.json. Environment template backend/.env.example with SECRET_KEY, DEBUG, ALLOWED_HOSTS, DATABASE_URL, and optional ANTHROPIC_API_KEY. Machine learning reproduction with python ml/generate_dataset.py followed by python ml/train_model.py requiring pandas and scikit-learn fixed environment.

Technology Choices. Main site is vanilla HTML with Tailwind CDN for file-protocol execution. Alternative scaffold is React with Vite and Tailwind with proxy of /api to Django on port 8000. Backend is Django REST on Render with Gunicorn. Machine learning is Random Forest at 89 percent with PCA and weighted fallback. Deployment is Vercel for frontend with Render for backend. Languages are ten dictionaries in frontend/src/i18n/*.json plus frontend dictionary in app.js with text-to-speech.

5.2 Summary

GuidanceAI requires only a standard laptop with a modern browser and open-source Python and Node stacks. No special hardware is needed. Offline-first design with lightweight SQLite, CDN libraries, and service-worker caching ensures laboratory and viva readiness, while Render with PostgreSQL and an optional Claude key unlocks full cloud capability.

6. MODULE DESCRIPTION

6.1 Modules

The system contains ten primary modules with five added professional extensions. The primary modules are: (1) Domain Explorer with Atlas 25, (2) Quiz Recommender with 15 questions, (3) Marks ML Filter with PCA and Random Forest, (4) Roadmap with 36-week handbook and four-year timeline, (5) Mini-Course with test, certificate, and XP, (6) Resource Hub with NPTEL, Coursera, and YouTube, (7) Vault per domain with 100 slots, (8) Chatbot with retrieval grounding and voice, (9) Dashboard for student, parent, and teacher, (10) Feedback with FAQ. The added extensions are Skill-Gap Radar, What-If Simulator, Parent Voice Report, Gamification with XP, streak, and badges, and Teacher Analytics with at-risk flags, plus Compare up to three tracks, command palette with Ctrl-K, PWA with dark mode and accessibility, toasts with bookmarks, and immersive 3D handbooks.

6.2 Key Modules

Domain Explorer. The `DOMAINS` array in `assets/js/app.js` defines 25 entries with identifier, name, category, tag such as Trending 2026, Evergreen, Critical, or Future, salary band, skills, and vault path. Intelligence and Data contains 6 tracks, Build and Experience contains 5, Core Systems contains 8, Frontier Technology contains 6. Features include category pills, live search, sort by featured with A to Z with salary and demand, compare checkbox with bottom bar allowing maximum three selections with side-by-side modal for salary, skills, and roadmap, command palette, and vault count badges.

Quiz and ML Filter. The quiz page implements 15 steps with progress bar and `localStorage` persistence. Backend `predict()` validates strictly, loads the pickle with scaler, PCA, and label encoder, builds an 11-feature vector from seven quiz signals plus four marks, transforms through scaler and PCA, predicts with Random Forest, derives confidence from `predict_proba` maximum multiplied by 100, and returns Top-3 with explanation. On any model failure, `weighted_predict()` computes values as 15 answers plus Mathematics divided by 25 plus Programming divided by 25, raw score as sum of value multiplied by weight multiplied by 25 per domain, normalised as value divided by maximum multiplied by 82 plus 8 capped at 95, sorted descending. Frontend `computePrediction()` mirrors the weights for parity.

Handbook Renderer. The `HANDBOOKS` object in `assets/js/handbooks.js` with 353 KB defines 25 handbooks each with 18 sections: overview, five pillars, eight curated sources with YouTube identifiers, economics context, interview pattern, ten previous interview questions, five coding problems with LeetCode identifiers, six-phase 36-week plan, salary bands from

AmbitionBox 2024 to 2025, top companies, government examinations including GATE, ISRO, NIC, and BARC, researcher track, and ethics. The renderer in `assets/js/domain.js` with 75 KB provides 3D hero through ECharts-GL neural field with distinct gradient per domain, skill-demand ECharts, GSAP ScrollTrigger timeline, salary bars, and vault theme.

Vault. Public authoritative storage is `public/vault/<slug>/` with four slots per domain for notes, videos, projects, and assignments, totalling 100 folders, visible in File Explorer. Backend private mirror is `uploads/domains/<slug>/` as Django `MEDIA_ROOT`. UI modal shows four dashed slots with Empty status and Add Slot button storing filename in `vault_<id>` `localStorage`. Bulk deployment drops real files with optional `index.json` manifest. Backend `vault_view()` lists live files, used by the chatbot for grounded answers. Documentation is in `docs/VAULT_MAP.md` and `FILE_MAP.md`.

Chatbot, Dashboard, Resource, and Feedback. Chat endpoint sanitises input to 500 characters, blocks prompt-injection phrases, resolves anaphora with history, builds context from keyword and alias matching plus vault live listing limited to six blurbs, attempts Claude 3.5 Sonnet with 700 tokens at temperature 0.4 when a key is present, otherwise applies TF-IDF cosine similarity over 26 documents with exact-match priority and daily-conversation handling for greetings and jokes. Dashboards provide student XP and roadmap progress, parent vernacular selector with voice function and print CSS, and teacher batch bar chart with at-risk rule where logic below 2 with Mathematics below 60 flags at-risk.

6.3 User Interface

Navigation is a sticky glass bar with skip-to-content link, logo with Atlas 25 badge, Ctrl-K hint, primary links to Home, Atlas, Handbook, Quiz, Roadmap, Resources, and Dashboard, language and role selectors, dark-mode toggle, pulsing Start Quiz call-to-action, mobile sticky quiz bar, and mobile drawer. Hero uses an indigo to cyan gradient with trust chips, headline, subtext, three buttons for quiz, explorer, and demo video, three statistics for students guided, domains, and languages, and a live prediction preview card with confidence bars, radar canvas, why-explanation box, and privacy note with a floating Telugu parent voice card. Trust bar lists NPTEL, Coursera, YouTube, models, and literature. Features section shows six cards. Atlas grid uses minimum-height cards with compare checkboxes and vault badges. Quiz call-to-action, handbook banner in slate-900 gradient, about preview, three testimonials, contact link, and four-column footer with explorer, users, tech with hidden developer details, and copyright complete the home page. Handbook page provides 3D hero, charts, timeline, and vault. All secondary pages load `../assets/js/app.js` and `../public/manifest.json`. Accessibility includes focus-visible rings, reduced-motion support, right-to-left layout for Urdu with Nastaliq font and line-clamp adjustments, lazy images, and ARIA labels.

6.4 Technology stack

Frontend uses HTML5, Tailwind CSS via CDN, Chart.js, ECharts with ECharts-GL, GSAP, Font Awesome, PWA service worker, and Web Speech API. Alternative frontend uses React 18, Vite with proxy of /api to port 8000, Tailwind CSS 3.4, and i18next with ten JSON dictionaries with chart.js. Backend uses Django 4.2, Django REST Framework with api_view decorators, CORS headers, Gunicorn, and SQLite with PostgreSQL readiness. Machine learning uses pandas, scikit-learn with StandardScaler, PCA, Random Forest with 100 trees, Decision Tree, KNN, TF-IDF with cosine similarity for chatbot fallback, and pickle artefacts containing model with scaler, PCA, encoder, and feature columns. Deployment and documentation use Vercel, Render, vercel.json, _headers, sitemap with robots and manifest, and ARCHITECTURE.md with ML_PIPELINE.md with VAULT_MAP.md with FILE_MAP.md with SECURITY.md.

6.5 Security Consideration

Content Security Policy restricts default sources to self with allowlisted Tailwind, jsDelivr, cdnjs, unpkg, Google Fonts, image CDNs, localhost APIs, and YouTube frames, with nosniff, strict-origin-when-cross-origin, HSTS, and X-Frame options through _headers. Prediction endpoint enforces a 10 KB request limit, strict validation of 15 integers from 1 to 4, marks clamping from 0 to 100, and ignores unexpected fields. Feedback and chat strip angle brackets, cap lengths at 500 and 1000 characters, clamp ratings from 1 to 5, require minimum text length, and block spam with more than three URLs, script tags, and prompt-injection phrases including ignore previous, system prompt, jailbreak, and drop table. Pickle loading validates that the model is a real scikit-learn object and not a mock string. No secrets are committed, with .env.example as template. Security contact is published through SECURITY.md with security.txt and .well-known/mirror. Privacy is first-party with no signup, localStorage persistence, no camera or biometric retention, and offline PWA operation.

7. IMPLEMENTATION

7.1 Implementation

Machine learning pipeline in `ml/`. The generator `ml/generate_dataset.py` synthesises 750 rows with 30 per domain across 25 domains and 15 columns including student identifier, seven quiz signals for logic, mathematics aptitude, creativity, security, cloud, data, and operating-system interest, four marks for Mathematics, Physics, Programming, and English, and a label. Domain-correlated logic ensures, for example, Artificial Intelligence has high logic with high mathematics and data interest, Human-Computer Interaction has high creativity with low mathematics, and Cybersecurity has high security with operating-system interest. The trainer `ml/train_model.py` applies StandardScaler, PCA from 11 to 10 components retaining 95 percent variance, and trains Random Forest with 100 trees, Decision Tree, and KNN with train-test split, writing `ml/metrics.json` with Random Forest 89 percent and cross-validation 82 percent, Decision Tree 64 percent with 68.8, KNN 82 percent with 77.4, a six-by-six confusion-matrix sample with 25 labels, and writing `ml/metrics.txt` with `model.pkl` of 1.5 MB copied to `backend/ml_service/`. Reproduction is `python ml/generate_dataset.py` followed by `python ml/train_model.py`. Standalone prediction is available through `backend/ml_service/predictor.py`.

Backend in `backend/`. Settings in `config/settings.py` configure CORS to the Vercel domain, SQLite, and `MEDIA_ROOT` as `uploads/domains/`. Routes in `config/urls.py` with `api/urls.py` map `/api/predict`, `/api/domains`, `/api/resources`, `/api/vault`, `/api/feedback`, and `/api/chat`. Models in `api/models.py` define `StudentProfile` with four marks, learning style, and personality, `QuizResponse` with ten scores, `Prediction` with recommended domain, confidence, and Top-3 JSON, `Domain` with description, skills, careers, and roadmap JSON, `Resource` with title, domain foreign key, type, language, difficulty, URL, and rating, and `Feedback` with name, role, text, and rating. Views in `api/views.py` with more than 600 lines define a 25 by 17 `WEIGHTS` matrix, `load_model()` with pickle validation, `weighted_predict()` with clamping and normalisation, `predict()` with strict validation and pickle-first logic, `domains()` with 25 static entries including salary and vault, `resources_view()` with NPTEL, Coursera, and YouTube mappings, `vault_view()` with live file listing, `feedback_view()` with sanitisation, `_build_guidance_context()` with keyword, alias, and vault enrichment limited to six blurbs, `_ml_fallback_answer()` with TF-IDF cosine similarity with exact-match priority and daily-conversation handling, and `chat()` with Claude integration and fallback. Execution is `pip install -r requirements.txt` with `python manage.py migrate` and `python manage.py runserver` on port 8000, with production as `gunicorn config.wsgi` on Render.

Frontend vanilla in `index.html` with `pages/` and `assets/js/`. The brain `assets/js/app.js` with 341 KB and 4617 lines defines the `DOMAINS` array with 25 entries, `resources` array, `computePrediction()` weighted logic with parity, renderers for grid, modal, radar, compare, palette, vault, bookmark, ten-language dictionary, XP, toast, dark mode, and chat speech functions. Knowledge `assets/js/handbooks.js` with 353 KB defines `HANDBOOKS` for 25 domains with 18 researched sections. Renderer `assets/js/domain.js` with 75 KB and 953 lines provides 3D hero, charts, timeline, and vault theme. Pages include about, quiz, explorer,

domain, dashboard, roadmap, resources, contact, and 404, each loading `../assets/js/app.js` with `../public/manifest.json`, verified after cleaning on 2026-09-22 with removal of `index_full.html`, empty Project file, six `_README.txt` stubs, `__pycache__`, `dist/`, and zero-byte `db.sqlite3`, and deduplication of `manifest`, `sitemap`, `service worker`, and `vault` to authoritative `public/` with root copies for Vercel and file-protocol discovery.

Frontend React in `frontend/`. Package manifest defines React 18 with `chart.js` and `i18next` with `dev`, `build`, `preview`, and `deploy` scripts. Vite configuration proxies `/api` to Django on port 8000 with output to `dist/`. Tailwind with PostCSS uses version 3.4. Entry `frontend/index.html` mounts `src/main.jsx` with `index.css`. Application shell `src/App.jsx` mirrors Atlas data. Components include `DomainCard` with icon, tag, salary, and vault count, `QuizEngine` with questions and marks ensemble predictor, `Chatbot` with retrieval and voice, and `RadarChart` with `Chart.js` radar. Pages include `Landing`, `Quiz`, and `Dashboard`. Internationalisation includes ten JSON dictionaries for English, Telugu, Hindi, Tamil, Kannada, Malayalam, Marathi, Bengali, Gujarati, and Urdu. Legacy `frontend/public/` mirrors authoritative `public/` with `manifest`, `icons`, `headers`, `security`, and `vault README`. Execution is `npm install` with `npm run dev` on port 5173 plus backend in parallel.

Vault, PWA, SEO, internationalisation, voice, and gamification. Manifest enables installable standalone mode with colour `#4F46E5` and shortcuts for `Quiz`, `Atlas`, `Handbook`, and `Vault`. Service worker provides cache-then-network offline fallback. Robots allow all with `sitemap` link. Sitemap lists 12 URLs including `home`, `explorer`, `quiz`, `domain identifiers`, `roadmap`, `resources`, `dashboard`, `about`, and `contact`. Headers enforce CSP, HSTS, and framing policy. Icons use data URIs at 192 and 512 px. Vault folders total 100 with optional `index.json` and `README`, with root copies for discovery and `.well-known/security.txt` mirror. Language switcher persists selection with Noto fonts, language-specific line heights from 1.8 to 2.2, right-to-left direction for Urdu, table and canvas direction fixes, and line-clamp expansion for Indic scripts. Parent voice uses speech synthesis with printable stylesheet. Experience points award 50 for quiz completion, 20 per lesson, and 5 per bookmark, with streak from daily last-visit tracking and badges for `Explorer`, `Polyglot`, and `Quiz Master`.

Integration testing verified the complete flow from `index.html` `quiz` to `/api/predict` or fallback to `Top-3` to `domain.html` `handbook` to `vault` addition to `Telugu` voice to `chatbot` comparison of AI versus Quantum to `dashboard XP`, on file protocol, localhost with port 8000, and Vercel preview.

8. RESULT ANALYSIS

8.1 Results Obtained

Machine learning accuracy from `ml/metrics.json` shows Random Forest at 89 percent with cross-validation 82 percent, K-Nearest Neighbours at 82 percent with 77.4, and Decision Tree at 64 percent with 68.8, with PCA from 11 to 10 components retaining 95 percent variance. The confusion-matrix sample shows a strong diagonal, for example 15 of 19 Artificial Intelligence cases correct with 4 confused with Computer Vision due to expected feature overlap, and 23 of 25 Cloud Computing cases correct. The balanced 750-row dataset with 30 per domain prevents majority-class bias. Viva evidence includes opening `ml/dataset.csv` with visible rows, `ml/metrics.json` with accuracy and matrix, and `backend/ml_service/model.pkl` loading in views.

Table 8.1 — Model comparison.

Model	Accuracy (percent)	Cross-validation (percent)	Remarks
Random Forest (100 trees, primary)	89	82	Best, deployed in pickle with fallback
K-Nearest Neighbours	82	77.4	Competitive, sensitive to scaling
Decision Tree	64	68.8	Interpretable baseline, overfits

Functional results confirm a 15-question quiz completion in approximately three minutes to Top-3 with radar and explanation, offline-online parity with identical Top-1 within rounding, Atlas search under 100 ms for 25 cards with instant sort, compare, and command palette, handbook 3D with ECharts rendering under two seconds on laboratory hardware, vault badge increment from 0 to N with toast confirmation, resource filtering by 25 domains with three types and language groups with instant bookmarking, chatbot exact-match for single-domain queries such as what is AI and HCI with vault live files listed, Claude path with sanitised response under 2000 characters with vault tip when a key is configured, Telugu text-to-speech verified in Chrome, PWA installation with offline quiz through service-worker cache, and dark mode with right-to-left and accessibility checks passing keyboard and screen-reader spot tests.

Figure references for the printed report: Figure 8.1 Hero with live prediction preview. Figure 8.2 Atlas grouped view with compare bar. Figure 8.3 Quiz radar with Top-3. Figure 8.4 What-if sliders. Figure 8.5 Handbook 3D hero with timeline. Figure 8.6 Resource Hub filter. Figure 8.7 Vault slots. Figure 8.8 Parent Telugu report with voice. Figure 8.9 Teacher analytics. Figure 8.10 Chatbot multilingual conversation. Figure 8.11 Metrics file with confusion matrix. Figure 8.12 Vercel with Render deployment logs.

User feedback from 324 reviews at 4.6 of 5 includes quiz radar preventing a wrong-track semester with Cybersecurity at 88 percent, Telugu voice helping parents understand without pressure, and teacher analytics flagging at-risk students early with batch satisfaction at 92 percent. Teacher distribution shows a 40 percent Cloud cluster leading to laboratory reallocation.

Comparison with existing work shows branch predictors at 78 percent with four labels versus 89 percent with 25 labels and explanation in this project, static portal articles versus interactive Top-3

with roadmap and vault, and keyword recommenders without visualisation versus radar with 3D handbook and gamification.

9. CONCLUSION AND FUTURE WORK

9.1 Conclusion

Personalised domain management is achieved by mapping quiz and marks to Top-3 with confidence and explanation rather than generic advice. Automated profiling eliminates manual branch lookup, with weighted fallback ensuring offline continuity. Timely roadmaps convert labels into action through 36-week plans, four-year timelines, mini-courses, tests, XP, and vaults. Voice with visual notifications combines radar, 3D charts, ten-language text-to-speech, and printable PDF to include parents. Multiple stakeholder support serves student, parent, and teacher with dedicated dashboards and 100 vault slots for cohort use. Progress tracking stores XP, streak, bookmarks, lesson status, and prediction history locally and in the database. Manual effort is reduced to a single double-click demo with one-command Vercel and Render deployment. The solution uses a standard laptop without IoT, RFID, camera biometrics, or dispenser hardware, preserving privacy. Overall effectiveness is demonstrated by integrating machine learning with full-stack engineering and multilingual design into a viva-ready and deployable counsellor that reduces hype-driven switching and involves families.

9.2 Future Work

Mobile application integration with React Native, push reminders, and offline SQLite synchronisation so students receive roadmap milestones on smartphones. Caregiver and mentor notifications through SMS and email when milestones are missed or at-risk flags trigger, helping guardians monitor elderly-dependent or first-generation learners. Cloud data storage with PostgreSQL and analytics dashboards for adherence reports and frequently missed phases. Improved machine learning with gradient boosting, calibration curves, and fairness audits across gender, region, and medium of instruction, plus large-language-model explanations grounded in handbooks. Study-time integration with Pomodoro tracking to XP automation. Proctored skill tests with auto-graded coding IDE embedding. Alumni and live-market API integration for real-time salary and vacancy updates.

9.3 Summary

GuidanceAI delivers an open, offline-capable, multilingual domain counsellor with Random Forest at 89 percent accuracy, Atlas 25 coverage, vault storage, and voice reporting, with clearly stated limitations and a credible path to mobile, cloud, and advanced-model futures. It is a foundation ready to present and straightforward to extend.

10. INDIVIDUAL TEAM MEMBER'S REPORT

10.1 Individual Objective

Member 1 objective is to own accurate recommendation, including dataset generation, model training, backend prediction API, and evaluation evidence. Member 2 objective is to own usable guidance, including Atlas interface, immersive handbooks with 3D visualisation, vault with resources and voice, and dashboards. The joint objective is a double-click demo with Vercel and Render deployment and viva evidence in a single repository.

10.2 Role of the Team Members

Team Member 1 — Machine Learning with Backend and Data Management. Generated and validated `ml/dataset.csv` with 750 rows and engineered 11 features. Built `ml/train_model.py` with scaler, PCA, and Random Forest, Decision Tree, and KNN comparison to `metrics.json` and `model.pkl`. Implemented `backend/api/models.py` with six tables and `backend/api/views.py` prediction, domains, resources, vault endpoints with `ml_service/predictor.py`. Managed database migrations, CORS with Unicorn deployment, and API validation with security hardening.

Team Member 2 — Frontend with Handbook and Vault with Voice. Built `index.html` with nine pages in `pages/` and `assets/js/app.js` quiz ensemble, Atlas rendering, internationalisation, radar, vault, and gamification with `handbooks.js` for 25 researched handbooks and `domain.js` for 3D, ECharts, and GSAP timeline. Built `frontend/src/` React scaffold with ten-language JSON dictionaries, vault UI, speech synthesis and recognition, dashboards, and chatbot interface. Curated the Resource Hub with NPTEL, Coursera, and YouTube mappings with `public/vault/` folders and PWA with SEO artefacts.

10.3 Contribution of Team Members

Area	Team Member 1	Team Member 2
Planning and requirements	Joint 7-week plan with 8 modules	Joint 7-week plan with 8 modules
Machine learning and data	Primary: dataset, training, 89 percent	Assisted: feature review, quiz testing
Backend and API	Primary: views, models, deployment	Assisted: fetch integration
Frontend and UX	Assisted: API contract	Primary: Atlas, handbook, voice
Vault and resources	Backend mirror in uploads	Primary: public vault with hub
Testing	API unit with confusion matrix	UI with viva flow, Telugu voice, offline
Documentation and report	Architecture with ML pipeline	UI with modules, results, screenshots
Presentation	Metrics with live prediction demo	Quiz to handbook to vault to voice demo

10.4 Summary

Both members jointly contributed to planning, requirement analysis, system architecture, module integration, testing, debugging, report preparation, and final presentation. Member 1 ensured the recommendation is correct with 89 percent evidence. Member 2 ensured the recommendation is understood and used through multilingual, visual, and actionable design. Collaboration integrated machine learning, backend, frontend, vault, and voice into a complete virtual counsellor.

REFERENCES

1. AI Journal of Multidisciplinary Research, 2025, AI-Based Career Counselling System using Decision Trees, branch-level prediction.
2. International Journal of Research and Analytical Reviews, 2022, Engineering Branch Prediction using Random Forest and KNN, accuracy near 78 percent.
3. Journal of Emerging Technologies and Innovative Research, 2024, E-Learning Recommender with NPTEL and Coursera keyword mapping.
4. Pedregosa et al., Scikit-learn: Machine Learning in Python, Random Forest, PCA, TF-IDF.
5. Django REST Framework documentation, api_view, CORS, Unicorn deployment.
6. React, Vite, Tailwind CSS documentation, single-page scaffold with proxy.
7. ECharts, ECharts-GL, GSAP ScrollTrigger documentation, 3D handbook visualisation.
8. Web Speech API, MDN, speech synthesis and recognition for ten-language voice reports.
9. NPTEL, Coursera, YouTube course catalogues, per-domain curation in resources array.
10. AmbitionBox salary bands 2024 to 2025, McKinsey AI economics context in handbooks.
11. Project evidence: ml/dataset.csv, ml/metrics.json, backend/ml_service/model.pkl, backend/api/views.py, assets/js/app.js, assets/js/handbooks.js, docs/ARCHITECTURE.md, docs/ML_PIPELINES.md, docs/VAULT_MAP.md, FILE_MAP.md.

End of Report — GuidanceAI Design Project 2025-2026. For viva, open in order: README.md, FILE_MAP.md, index.html with quiz, pages/domain.html with id AI, public/vault/ai/notes/, ml/dataset.csv with metrics.json, backend/api/views.py, assets/js/handbooks.js.